Crypto Market ELT — the complete walkthrough

Everything we built, every concept we used, explained slowly with real examples from the project. Written as a study companion — no assumed knowledge beyond "I know some SQL and some Python".

Project: github.com/vinn27/crypto-market-elt · dashboard: crypto-market-elt.netlify.app · built 9–12 October 2026

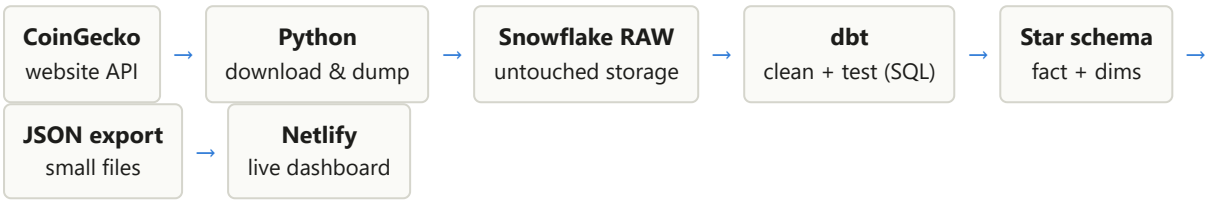
How to read this document: Chapters 1–9 follow the data's journey from a public website to a live dashboard — the same order the pipeline actually runs in. Chapter 10 covers the dashboard-design lessons, Chapter 11 is interview rehearsal, and Chapter 12 is a map of every file in the repo. Whenever we hit a problem during the build, it appears in a red box like the one below — those boxes are the most valuable parts, because interviewers love asking "what went wrong?".

> **Real example of a problem box**
>
> Every red box in this document is a genuine failure from this build — the actual error message, why it happened, and how we fixed it. Nothing is invented.

1 · The big picture — what we built and why

1.1 The one-paragraph story

Every morning at 06:43 IST, GitHub (for free) lends us a small computer. It runs a Python script that downloads crypto-market data from a public website. The script dumps that data, completely untouched, into a cloud data warehouse called **Snowflake**. A tool called **dbt** then cleans and reshapes that data inside Snowflake using SQL, checks it with 15 automated tests, and builds a small "star schema". Finally, the numbers are exported to small files and published as a live, animated dashboard on **Netlify**. No human presses any button. The whole thing costs ₹0.



1.2 Each tool's job, with an everyday analogy

Tool	Its job	Everyday version
CoinGecko API	The data source — a website that gives data instead of pages	A shopkeeper who answers questions about today's prices
Python + requests	Fetches data and loads it — nothing else, by rule	The delivery boy: picks up the groceries, puts them in the fridge, doesn't cook
Snowflake	Cloud data warehouse — stores and queries analytics data	The fridge + a rented kitchen you pay for only while cooking
dbt	Runs SQL transformations, tests, and documentation	The chef following a recipe book where every recipe is a small SQL file
GitHub Actions	Runs the whole thing on a schedule, on GitHub's computers	An alarm clock + a robot that follows one recipe file every morning
Netlify + CDN	Serves the dashboard worldwide for free	A shop window: anyone can look, nobody enters the kitchen

1.3 Why we chose this stack (the honest reasons)

- **Snowflake + dbt** appear together in most 2026 data-engineering job descriptions — and this project gave hands-on time with both.
- **Everything free, no credit card:** Snowflake trial (30 days, email only), GitHub Actions free minutes for public repos, Netlify free tier, CoinGecko public API without a key.
- **ELT on purpose:** my earlier stock-market project was classic **ETL** (PySpark cleaned data *outside* the database, then loaded MySQL). This project is the opposite pattern — load raw first, clean inside the warehouse — so together the two projects show both industry patterns.

ETL vs ELT — the core idea of this whole project

ETL (old way): Extract → **T**ransform (clean in Python/Spark) → Load. If your cleaning logic is wrong, the raw data is already gone — you must re-download everything.

ELT (our way): Extract → Load raw *first* → **T**ransform inside the warehouse. Raw data is preserved forever. Wrong logic? Fix the SQL, re-run, done — history is still sitting there untouched in the RAW layer.

Modern warehouses made this possible: Snowflake is so fast at SQL that copying data out to Spark just to clean it stopped making sense for analytics work.

2 · The source — understanding CoinGecko's API

2.1 What an API actually is (in simple words)

When you open coingecko.com in a browser, you get a pretty page meant for human eyes. The same company also offers a **second door** for programs: you send a URL, and instead of a pretty page you get raw data back — usually as **JSON**. That second door is the API. Free, no key needed for basic use, with one catch we'll meet soon: **rate limits** — you may only knock so many times per minute.

2.2 The two doors we used

Endpoint (URL)	What it gives	Where it lands
`/coins/markets`	One snapshot: top-50 coins, current price, market cap, 24h volume, ranks…	`RAW.COINS_MARKETS`
`/coins/{id}/market_chart`	History: daily price, market cap and volume for ~90 days, one coin per call	`RAW.COIN_MARKET_CHART`

2.3 What the data actually looks like

We always inspect the source *before* writing code. The snapshot endpoint returns a list of objects — here's the first one, trimmed (26 fields per coin; numbers and text mixed, some fields sometimes `null`):

```
{
  "id": "bitcoin",
  "symbol": "btc",
  "name": "Bitcoin",
  "current_price": 82506,
  "market_cap": 1658126145503,
  "market_cap_rank": 1,
  "total_volume": 39828442115,
  "high_24h": 83111,
  "low_24h": 80427,
  "price_change_percentage_24h": -0.56848,
  "ath": 126512,        // all-time high
  "ath_date": "2026-01-19T04:15:32.000Z",
  "last_updated": "2026-10-09T08:15:32.104Z"
  ...
```

The history endpoint has a trickier shape — **three parallel arrays**, where each item is a pair of `[milliseconds-timestamp, value]`:

```
{
  "prices":        [[1790726400000, 83640.10], [1790812800000, 83576.13], ...],
  "market_caps":   [[1790726400000, 1680405670009.9], ...],
  "total_volumes": [[1790726400000, 29007802596.08], ...]
}
```

"Parallel arrays" means: position 0 of `prices`, position 0 of `market_caps` and position 0 of `total_volumes` all describe the **same day**. To turn that into normal rows, our Python *zips* them together — that's the one piece of "unboxing" the extract does (Chapter 3.4).

Problem 3 — the source lies about completeness

We asked for 90 days of history for the top-10 coins. Nine coins returned 91 points each. `figure-heloc` returned only **81** — the coin is simply younger than 90 days. Also, those "91 points for 90 days"? The API gives one point per day at 00:00 UTC *plus* one live "now" point — so two points share today's date. Both quirks are normal source messiness: the pipeline must absorb them without breaking (Chapters 5 and 7 show how the schema and tests handle exactly this).

3 · Extract + Load — the Python part, file by file

The code lives in `pipelines/extract_load.py`. One rule governs the whole file: **Python only downloads and dumps. No cleaning, no renaming, no business logic.** Everything clever happens later, in SQL, inside the warehouse.

3.1 The fetch (with polite retries)

```
def get_with_retry(url, params):
    for attempt in range(1, MAX_RETRIES + 1):          # up to 5 tries
        r = requests.get(url, params=params, timeout=30)
        if r.status_code in (429, 503, 504) and attempt < MAX_RETRIES:
            wait = int(r.headers.get("Retry-After", 30)) # server says how long to wait
            print(f"rate-limited ({r.status_code}): backing off {wait}s")
            time.sleep(wait)
            continue                                   # try again after waiting
        r.raise_for_status()                           # any other error -> crash loudly
        return r.json()
```

Why this exists — HTTP `429 Too Many Requests` means "you're knocking too fast, come back later". Many APIs include a `Retry-After` header telling you exactly how long to wait. Reading it and waiting is called **backoff**. Between history calls we also sleep 10 seconds out of politeness.

> **Problem 2 — blocked mid-run**
>
> Our very first run loaded 50 snapshot rows + 6 coins of history, then died: `429 Client Error: Too Many Requests for url .../coins/solana/market_chart`. Our 7-second pause was too fast for the free tier. Fix = the retry function above (plus a longer pause). The re-run survived an actual 429 mid-flight: it printed "backing off 5s" and continued to green. Lesson: a bigger sleep alone is a guess; **retry-with-backoff is the pattern** — every serious ingestion tool has it built in.

3.2 The load — and what "raw" means in practice

"Land it as-is" has precise rules we followed in the table design:

- JSON numbers → `FLOAT` columns (the value passes through untouched).
- JSON text → `VARCHAR`, **even dates**. Casting is transformation, and transformation belongs to dbt. So `ath_date` lands as the string `"2026-01-19T04:15:32.000Z"` — dbt turns it into a real timestamp later.
- Column names copy the API field names *exactly* (`id`, not `coin_id`) — renaming is also transformation.
- One extra column the API never sends: `loaded_at`, filled by Snowflake itself (`DEFAULT CURRENT_TIMESTAMP()`) — a stamp saying when this row arrived. It powers everything in Chapter 5.

```
CREATE TABLE IF NOT EXISTS RAW.COIN_MARKET_CHART (
    coin_id      VARCHAR,
    price_date   DATE,        # derived from ms-timestamp, only to unpack the array
    price        FLOAT,
    market_cap   FLOAT,
    total_volume FLOAT,
    loaded_at    TIMESTAMP_TZ DEFAULT CURRENT_TIMESTAMP()
)
```

> **Problem 1 — invalid identifier 'ID'**
>
> First insert crashed with `000904: SQL compilation error: invalid identifier 'ID'`. Cause: my first table design "helpfully" renamed `id` to `coin_id`, but the INSERT listed the API's field names. Column list ≠ table shape. Fix: keep API names in RAW; `stg_coins` does `id AS coin_id` later — which is exactly where renaming belongs. (Side-lesson: `CREATE TABLE IF NOT EXISTS` won't fix a wrong existing table — we had to DROP it first.)

3.3 Unpacking the parallel arrays

```
for (ts_ms, price), (_t, market_cap), (_t2, total_volume) in zip(
        data["prices"], data["market_caps"], data["total_volumes"]):
    price_date = datetime.fromtimestamp(ts_ms / 1000, tz=timezone.utc).date()
    rows.append((coin_id, price_date, price, market_cap, total_volume))
```

`zip()` walks the three arrays together; converting `1790726400000` (milliseconds) into the date `2026-09-30` is *structural unpacking*, not transformation — the values themselves are untouched.

3.4 Inserting into Snowflake

We use the connector's `executemany()` with one INSERT — the Snowflake driver optimizes it into a single bulk multi-row insert. At our scale (~1,000 rows) this takes seconds. The production-scale path (millions of rows) would be: write a file to a Snowflake **stage**, then `COPY INTO` — same warehouse, same ELT story, different loader. Good line for interviews: "I know executemany was fine at 1K rows and where I'd switch to COPY."

3.5 Idempotency — why re-running never breaks anything

RAW tables are **append-only**: we never UPDATE or DELETE. Run the script twice? You get two copies of bitcoin's row, each stamped with its own `loaded_at`. That sounds like a bug; it's actually a design choice — the cleaning layer decides which copy wins ("the newest one"), so loading is always safe. "Safe to re-run" is what the word **idempotent** means.

4 · Snowflake — the warehouse, explained properly

4.1 The object model (what contains what)

```
ACCOUNT      VZJVHYS-IC67204          # the tenant you log into
 └─ DATABASE   CRYPTO_DB                 # like a MySQL database
     ├─ SCHEMA RAW                       # a namespace inside the db
     ├─ SCHEMA STAGING                   # our three medallion layers…
     └─ SCHEMA MARTS                     # …as three schemas
WAREHOUSE    COMPUTE_WH                 # compute, NOT storage (see 4.3)
ROLE / USER                             # permissions / login identity
```

MySQL hides the schema layer; Snowflake makes it a real object — and we exploited that: our bronze/silver/gold layers are schemas, so the layers are visible structure, not naming conventions. (Every database also ships with `INFORMATION_SCHEMA` and a default empty `PUBLIC` — ignore both.)

> **Two names, one account**
>
> Our login URL is `VZJVHYS-IC67204.snowflakecomputing.com` (the "org-account" name, used in configs), but `SELECT CURRENT_ACCOUNT()` returns `WM80161` (the internal "locator"). Both point at the same account — a five-minute confusion worth knowing about in advance.

4.2 Warehouse vs database — the real difference

	MySQL (database)	Snowflake (warehouse)
Built for	Apps: many small reads/writes of single rows	Analytics: sums and joins over millions of rows
Storage layout	Rows together (grab one row fast)	**Columns together** (sum one column without touching others)
Compute	The same server that stores the data	Separate rented clusters ("virtual warehouses")
Scaling	Bigger box	Pick a size (XS→6XL, each ≈ 2× the last) or add a second one
Billing	Server runs 24/7	Credits burned **only while the warehouse runs**

4.3 The rented kitchen — storage/compute separation

The single most important Snowflake idea. Your data sits in cheap cloud storage (the fridge — always there, billed by GB). Queries run on a **virtual warehouse** — a cluster of machines you rent by the **second** (the kitchen). Even `SELECT 1` briefly wakes an XS warehouse. Stop querying for 60 seconds and **auto-suspend** switches it off — billing stops. Next query auto-resumes it (you feel this as a few seconds of cold start). Interview line: "*storage-compute separation means I scale and pay for compute independently — my XS auto-suspends when idle.*"

4.4 Cost governance — what we set and why

```
ALTER WAREHOUSE COMPUTE_WH SET
    AUTO_SUSPEND = 60                   # sleep after 60 idle seconds
    AUTO_RESUME  = TRUE                 # wake on next query
    STATEMENT_TIMEOUT_IN_SECONDS = 600  # kill any query older than 10 min
```

The trial gives $400 of credits. A few small queries a day would never exhaust them — but an idle warehouse that never sleeps, or one runaway query, could. This ALTER (added to `setup_snowflake.py`, so it's reproducible from the

repo) makes bleeding credits *structurally impossible*. This is what "governance" means in practice: not hoping, configuring.

5 · dbt — the transformation layer, properly this time

5.1 What a dbt project actually is

Expectations of a big framework; reality: **one YAML file plus SQL files**. `dbt_project.yml` names the project, points to a connection profile, and sets per-folder defaults. Every `.sql` file under `models/` is a **model** — a single SELECT that dbt turns into a view or table. That's the whole magic. Ours:

```
models:
  crypto_elt:
    staging:
      +materialized: view     # cleaning layer: zero copied data
      +schema: staging
    marts:
      +materialized: table    # presentation layer: physically built
      +schema: marts
```

5.2 profiles.yml — secrets without secrets

```
crypto_elt:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: "{{ env_var('SNOWFLAKE_ACCOUNT') }}"
      user:     "{{ env_var('SNOWFLAKE_USER') }}"
      password: "{{ env_var('SNOWFLAKE_PASSWORD') }}"   # read from env, never stored
      warehouse: COMPUTE_WH
      database: CRYPTO_DB
```

`env_var()` means the file contains *zero* credentials — it can be committed, and the same file works later on GitHub Actions, which injects the same variables from its vault. Locally the values come from `.env` (git-ignored).

5.3 sources vs ref — how the dependency map is born

```
# sources.yml — tables Python built; dbt only points at them
sources:
  - name: coingecko
    database: CRYPTO_DB
    schema: RAW
    tables:
      - name: coins_markets
      - name: coin_market_chart
```

In a model: `{{ source('coingecko', 'coins_markets') }}` = "read the raw table dbt didn't build". `{{ ref('stg_coins') }}` = "read another model dbt *did* build — and build it before me". dbt collects every ref into the **DAG** (directed acyclic graph — a to-do list that can never loop). When we run `dbt build`, dbt itself figures out: sources → staging → tests → marts → tests. Nobody tells it the order.

5.4 Staging model 1 — stg_coins, block by block

```
with source as (
    select * from {{ source('coingecko', 'coins_markets') }}
```

```
),
deduped as (
    select *,
        row_number() over (
            partition by id                 # one group per coin
            order by loaded_at desc         # newest load first
        ) as rn
    from source
)
select
    id              as coin_id,             # ← renaming happens HERE, not in RAW
    upper(symbol)   as symbol,
    name            as coin_name,
    current_price, market_cap, market_cap_rank, total_volume,
    ...
    try_to_timestamp_tz(ath_date) as ath_date,   # string → real timestamp HERE
    try_to_timestamp_tz(last_updated) as source_last_updated,
    loaded_at
from deduped
where rn = 1                                # keep only the newest row per coin
```

The dedupe, walked through with our real duplicate. RAW contained two bitcoin rows (two pipeline runs):

id	current_price	loaded_at	rn
bitcoin	82,512	09:14 (later run)	**1** ← kept
bitcoin	82,489	07:28 (earlier run)	2 ← dropped

`row_number()` numbers rows inside each coin-group, newest first; `where rn = 1` keeps one. Ethereum's 2 rows become 1, all 50 coins the same. That's "latest load wins" — the payoff of append-only RAW. And `try_to_timestamp_tz` converts the ISO string to a real timestamp, returning NULL instead of crashing on a malformed value — defensive by design.

5.5 Staging model 2 — stg_market_chart, different grain

Same pattern, one difference that matters: the **partition** is `(coin_id, price_date)` — one row per coin *per day*, not per coin. Remember the "91 points but 90 days" quirk? The live "now" point shares today's date with today's daily point; both arrive as separate RAW rows; this dedupe collapses them. Nothing was lost — the grain rule did exactly its job.

> **Grain — say it out loud before writing any model**
>
> **Grain** = "what does ONE row in this table mean?" stg_coins: one row per coin. stg_market_chart: one row per coin per day. Deciding this first makes every dedupe, join and test obvious. In interviews, answering "what's your grain?" crisply signals real modeling experience.

5.6 Materializations — view vs table

A **view** is a saved query: a window over the source, copies nothing, always fresh, costs nothing to store. A **table** is a photo: physically built once, fast to query, frozen until rebuilt. We made staging = views (cleaning logic should always reflect RAW as-is) and marts = tables (stable and fast for the dashboard).

Small gotcha we dodged: schema naming

With folder config `+schema: staging`, dbt's default names the schema `<target>_staging` → "STAGING_STAGING". A tiny override macro (`macros/generate_schema_name.sql`, 6 lines) makes custom names pass through exactly as written — clean STAGING and MARTS.

6 · The star schema — the warehouse's finished product

6.1 Fact vs dimension, in one picture

One **big** table of numbers (the **fact**), small tables of descriptions (**dimensions**) around it, joined by IDs — draw the lines and it looks like a star ✳. The school-register analogy: the big sheet says "roll 14, present"; the small sheet says "roll 14 = Vineet". You never write the full name 500 times.

Table	Type	Grain (one row = ...)	Columns
fact_market_snapshots	fact	one coin, one day	coin_id, price_date, price, market_cap, total_volume
dim_coins	dimension	one coin	coin_id, symbol, name, rank, supply, ATH/ATL + dates
dim_date	dimension	one calendar day	date_day, day_name, day_of_week, month_name

Real sample rows (from the actual build):

coin_id	price_date	price	market_cap	total_volume
bitcoin	2026-10-11	82,512	1.65 T	39.8 B
ethereum	2026-10-11	2,502	301 B	15.2 B
bitcoin	2026-10-10	81,940	1.64 T	41.1 B

6.2 Why not one big flat table?

Tempting: one table with coin name, symbol, rank, day name repeated on every row. Costs: (1) storage bloat — "Bitcoin" written 90 times per coin; (2) drift — fix a typo in one place, miss 89; (3) slow filters — text filters instead of integer joins. The star keeps descriptions written once and joins them by ID. This trade-off is called **normalization for dimensions**, and it's the heart of dimensional modeling.

6.3 dim_date — the dimension that earns its keep

```
with dates as (
    select distinct price_date as date_day
    from {{ ref('stg_market_chart') }}      # whatever days exist in the data
)
select date_day,
       dayname(date_day)         as day_name,     # 'Saturday'
       dayofweek(date_day)       as day_of_week,
       monthname(date_day)       as month_name
from dates
```

A date like `2026-10-11` knows its number, not its meaning. dim_date attaches the meaning — and it **grows itself**: when new days arrive in the data, new dim_date rows appear automatically (90 → 92 rows during the build, zero changes needed).

6.4 The payoff query — weekday effect, line by line

```
WITH daily AS (
    SELECT price_date, price,
           100.0 * (price - LAG(price) OVER (ORDER BY price_date))
                 / LAG(price) OVER (ORDER BY price_date) AS day_change_pct
    FROM fact_market_snapshots
```

```
    WHERE coin_id = 'bitcoin'
)
SELECT d.day_name,
       ROUND(AVG(daily.price), 0)        AS avg_price,
       ROUND(AVG(daily.day_change_pct), 2) AS avg_change
FROM daily
JOIN dim_date d ON daily.price_date = d.date_day
GROUP BY d.day_name
```

- `LAG(price)` = "the previous row's price" — so `day_change_pct` is yesterday→today % move. One window function replaces a self-join.
- `JOIN dim_date` is the whole point: it's what makes "Saturday" a groupable thing. Without dim_date this query is impossible — a raw date can't tell you its weekday.
- Result on our real data: Friday averaged **+0.58%** moves, Wednesday **+0.05%** — Friday was ~11× livelier than Wednesday in this window.

7 · Testing — the 15 guards

7.1 The idea: tests hunt for bad rows

A dbt test is a SELECT whose job is to find rows that *should not exist*. Zero rows returned = pass. Any rows = the build fails loudly, in red, before bad numbers ever reach a dashboard. Tests run on every `dbt build` — ours run nightly on GitHub, so a data problem is caught within hours, automatically.

7.2 Generic tests — one line each

```
columns:
  - name: coin_id
    data_tests:
      - not_null                # any row where coin_id is empty? fail
      - unique                  # same coin twice in a dimension? fail
      - relationships:          # a fact row pointing at a coin that
          arguments:            # doesn't exist in dim_coins? fail
            to: ref('dim_coins')
            field: coin_id
```

Applied across staging + marts: not-null on every key, unique coin IDs, unique dates, relationships from the fact to *both* dimensions. Think of `relationships` as referential integrity for analytics — the "broken link" detector.

7.3 Custom tests — just SQL files with an opinion

```
# tests/assert_prices_are_positive.sql
select * from {{ ref('fact_market_snapshots') }}
where price is null or price <= 0       # finds bad rows; empty = pass
```

```
# tests/assert_no_duplicate_fact_rows.sql
select coin_id, price_date, count(*)
from {{ ref('fact_market_snapshots') }}
group by 1, 2
having count(*) > 1                     # any coin-day appearing twice? fail
```

Why "price > 0" matters: a zero/negative price can only mean broken source data — and it would poison every average downstream. Cheap to check, expensive to miss.

7.4 The full scoreboard

Check	Count	Catches
not_null (keys, dates, prices)	7	rows that lost their identity in loading
unique (coin IDs, calendar days)	3	sneaky dimension duplicates
relationships (fact → both dims)	2	facts joined to nothing
custom: no duplicate coin-days	1	double-loads reaching the marts
custom: prices positive	1	garbage values
staging-level key checks	1	dedupe bugs early in the chain

How the quirks from Chapter 2 got absorbed

`figure-heloc`'s missing 9 days? Not a test failure — fewer rows per coin is valid; only *duplicate* or *null* rows fail. The doubled "now" point? Collapsed by the staging grain before tests ever ran. Good tests check invariants ("what must always be true"), not exact counts ("how many rows I expect today") — exact-count tests break every time data grows.

8 · Orchestration — GitHub Actions, the robot that runs everything

8.1 The recipe file

A workflow is one YAML file in the repo (`.github/workflows/daily-pipeline.yml`) that says: *when* to run and *what* to do. When triggered, GitHub rents us a fresh Ubuntu computer, checks out our code, and follows the steps:

```yaml
on:
  schedule:
    - cron: "13 1 * * *"      # m h dom mon dow — 01:13 UTC daily = 06:43 IST
  workflow_dispatch: {}        # + a manual "Run workflow" button

jobs:
  pipeline:
    runs-on: ubuntu-latest
    env:                       # secrets injected here, invisible in logs
      SNOWFLAKE_ACCOUNT: ${{ secrets.SNOWFLAKE_ACCOUNT }}
      SNOWFLAKE_PASSWORD: ${{ secrets.SNOWFLAKE_PASSWORD }}
      NETLIFY_AUTH_TOKEN: ${{ secrets.NETLIFY_AUTH_TOKEN }}
    steps:
      - uses: actions/checkout@v4            # get the code
      - uses: actions/setup-python@v5        # python 3.12
      - run: pip install requests python-dotenv snowflake-connector-python dbt-snowflake
      - run: python pipelines/extract_load.py        # 1 fetch + load
      - run: dbt build                               # 2 clean + marts + 15 tests
      - run: python pipelines/export_dashboard_data.py  # 3 export JSON
      - run: |                               # 4 commit fresh data as "pipeline-bot"
          git add dashboard/data
          git commit -m "data: refresh $(date -u +%F)"
          git push
```

8.2 Secrets — the vault habit

Credentials live in exactly two places: local `.env` (git-ignored, never committed — verified with `git ls-files` before the repo went public) and GitHub's encrypted **Secrets vault** (repo Settings → Secrets), which pastes values into the runner as environment variables at runtime — visible to the code, never printed in logs, never in the repo.

> **Problem 5 — the morning the cron didn't come**
>
> Oct 12, ~07:00 IST: the dashboard still said "as of Oct 11". The Actions history showed **no scheduled run at all** — only our manual runs. Nothing was broken: GitHub cron is **best-effort** and skips/delays under platform load. Fixes: ran the pipeline manually (idempotent, so a late run loses nothing — it just fetches the latest data), moved the cron off the crowded `:05` minute to `01:13 UTC`, and kept the manual button as the always-available fallback. Production answer to this problem: an orchestrator with SLA monitoring and alerts.

> **Problem 6 — my own YAML bug, found the same morning**
>
> The deploy step used `if: ${{ secrets.X != '' }}` — but GitHub **forbids the secrets context inside step-level if conditions**. The workflow file failed to parse ("Unrecognized named-value: 'secrets'"), which would have silently killed *every* future run, cron included. The standard fix: expose the secret in the **job-level env** (allowed), and test `env.X != ''` in the step. Classic gotcha, now burned into memory.

8.3 What Airflow would add (the interview upgrade path)

GitHub Actions is orchestration-lite: one schedule, sequential steps, a web page of run history. **Airflow** is the industrial version: per-task retries with policies, *backfills* (rerun the past 30 days after a bug fix), *sensors* (wait until source data exists), SLA alerts, a graph UI of every DAG run. Our pipeline maps 1:1 — each workflow step becomes an Airflow task. That's the stated next step for this project, and it's a great interview line.

9 · Serving — from warehouse to live dashboard, for free

9.1 The problem with "dashboard queries the database"

Two issues: (1) credentials would have to live *in the browser* — anyone could steal them; (2) every visitor costs warehouse compute. Portfolio sites get crawled, shared, opened by recruiters at 2 AM — live queries are the wrong shape.

9.2 Our shape: export once, serve statically

The nightly pipeline's export step (`export_dashboard_data.py`) runs a handful of SELECTs against staging + marts and writes four small files — `overview.json`, `history.json`, `top_coins.json`, `weekday.json`. The workflow commits them (that's the nightly `data: refresh` commit — the repo's history doubles as a visible data trail), and the site deploys to Netlify's **CDN** — a worldwide network of servers that hand out static files fast and free.

> **The beautiful property**
>
> **Visitors never touch Snowflake.** No credentials in the browser, no per-visit cost, page loads in milliseconds from the nearest CDN node. The warehouse does its few seconds of work once per day, then sleeps. Freshness = daily — exactly the cadence the pipeline runs at, so nothing is lost.

9.3 The dashboard itself

A single static page (no framework) with the **ECharts** chart library: KPI tiles with count-up animation, a 90-day price chart with a coin selector, a measure toggle (price / market cap / volume), an "indexed to 100" scale, top-10 bars, the weekday-effect chart with its own coin selector, and a sortable top-20 table — plus the documentation page. All data comes from those four JSON files.

10 · Chart sense — three rules we followed (and why they matter)

10.1 Never two y-axes on one chart

Bitcoin ≈ $82,000; a small coin ≈ $2.50. Plot both on one dollar axis and the small coin is a flat line at the floor. The dishonest shortcut is a second y-axis — but the two scales' alignment is arbitrary, so the chart quietly invents correlations that don't exist. The honest fix is **indexing**:

	Day 1	Day 90	Raw change	Indexed (Day 1 = 100)
Bitcoin	$80,000	$84,000	+5%	100 → **105**
Small coin	$2.50	$3.50	+40%	100 → **140**

Rebase every series to 100 at the window start and growth becomes comparable on **one** axis. Our dashboard's "Indexed to 100" toggle does exactly this.

10.2 Validate colors — don't eyeball them

About 8% of men see color differently; a red/green pair can be identical to them. Before shipping we ran the chart palette through a colorblind-safety validator script — it flagged the worst pair as "floor band" (borderline), which is legal only with backup encoding. So the dashboard ships: legend chips with color dots, tooltips that name every series, and a full table view — identity never rides on color alone.

10.3 Every chart gets a table

Tooltips are enhancement, not gatekeeping: any value visible on hover is also visible in the sortable table — including to screen readers and anyone who just prefers numbers. Small discipline, big credibility.

> **Problem 7 — the buttons that did nothing**
>
> Dashboard v1 shipped range buttons (7D/30D/90D) and a USD/Indexed toggle that *looked* clickable but had no event listeners — decorative. Found during the v2 upgrade and fixed with a proper `wireControls()` function. Lesson that survives any tech stack: **click your own controls before shipping.**

11 · Interview rehearsal

Q: Walk me through your pipeline.
CoinGecko's free API → Python extracts and loads raw into Snowflake (append-only, `loaded_at`-stamped) → dbt rebuilds staging (latest-load-wins dedupe, rename, cast) and a marts star schema (fact = one row per coin per day; coin + calendar dimensions) → 15 tests gate every build → GitHub Actions runs it nightly, exports JSON, commits it, and deploys the dashboard to Netlify. Fully autonomous — the repo shows a data-refresh commit every morning.

Q: Why ELT over ETL?
Raw data is preserved — a wrong transform costs a re-run, not the data. And modern warehouses make in-warehouse SQL cheap. I've built both patterns: my stock-market project is PySpark ETL into MySQL; this one is Snowflake/dbt ELT.

Q: How do you handle duplicates?
Duplicates across loads are *expected* — RAW is append-only by design. Staging dedupes with `row_number() over (partition by id[, date] order by loaded_at desc)` keeping rn = 1: latest load wins. A custom test also fails the build if a duplicate ever reaches the fact table.

Q: What's your fact table's grain?
One row per coin per calendar date. Deciding that up front fixed the dedupe key, the uniqueness tests, and even absorbed the API's quirk of sending 91 points for 90 days (the live "now" point shares today's date and collapses).

Q: Explain Snowflake's architecture in one breath.
Storage and compute are separate: data in cheap columnar storage; queries on virtual warehouses rented per second. Mine is XS, auto-suspends after 60 idle seconds, auto-resumes, statements time out at 600s — cost governed by configuration, not hope.

Q: What does dbt give you over plain SQL scripts?
Version-controlled models with automatic dependency ordering (refs build the DAG), testing baked into every run, generated docs and lineage. Software-engineering practice — modularity, CI, documentation — applied to SQL.

Q: A test failed at 3 AM. What happens?
The run goes red and stops after that model's tests — marts don't rebuild with bad data, the dashboard keeps yesterday's (valid) JSON, and the Actions history shows which check failed. Because RAW is append-only, the fix is: correct the model or wait for the source to behave, re-run — no cleanup needed.

Q: The API rate-limits you. Then what?
Happened in the first run — a 429 mid-history. The loader retries with backoff, honouring the server's Retry-After header, up to five times, with a polite pause between calls. It survived a real 429 and still finished green.

Q: How are credentials handled?
.env locally (git-ignored — verified via git ls-files before going public), GitHub Secrets for CI (injected as env vars), `env_var()` in dbt's profile so the config file itself is committable, revocable tokens for deploys. Anything pasted anywhere is rotated — we rotated the Snowflake password during the build as practice.

Q: How does the dashboard get data?
The nightly run exports four small JSONs and deploys them with the static site to a CDN. Visitors read static files — no warehouse connection, no credentials in the browser, zero per-visit cost.

Q: What broke during the build?

Six real things: an `invalid identifier` from renaming a column in RAW (fix: rename in staging, not RAW); a 429 rate-limit (fix: retry with backoff); a coin with 81 of 90 days (absorbed by schema design); GitHub cron skipping a morning (manual run + moved schedule + idempotency means nothing lost); `secrets` in a step-if which broke workflow parsing (fix: job-level env); and dashboard buttons without listeners (fix: click your own controls).

Q: What's next for this project?

Airflow instead of cron-style Actions (retries, backfills, sensors, SLA alerts); incremental fact builds instead of full refresh; dbt snapshots if coin attributes start changing (SCD); more coins behind a proper rate-limit budget; alerts when a run misses its window.

12 · Repo map — every file, one line each

Path	What it is
`pipelines/extract_load.py`	Fetch CoinGecko (retries + backoff), land raw in Snowflake
`pipelines/setup_snowflake.py`	Idempotent DDL: database, 3 schemas, warehouse governance
`pipelines/export_dashboard_data.py`	Marts → four dashboard JSON files
`pipelines/test_connection.py`	Tiny sanity check: can Python reach Snowflake?
`dbt/dbt_project.yml`	Project name, profile, per-folder materializations
`dbt/profiles.yml`	Connection via env vars — no secrets inside
`dbt/models/staging/sources.yml`	Declares the two RAW tables as dbt sources
`dbt/models/staging/stg_coins.sql`	Snapshot cleaner: dedupe, rename, cast
`dbt/models/staging/stg_market_chart.sql`	History cleaner: one row per coin per day
`dbt/models/marts/dim_coins.sql`	Coin descriptions, one row per coin
`dbt/models/marts/dim_date.sql`	Self-growing calendar dimension
`dbt/models/marts/fact_market_snapshots.sql`	Daily numbers: price, cap, volume
`dbt/models/*/schema.yml`	Generic tests (not_null, unique, relationships)
`dbt/tests/*.sql`	Custom tests: no duplicate facts, positive prices
`dbt/macros/generate_schema_name.sql`	Pass-through schema naming (no STAGING_STAGING)
`.github/workflows/daily-pipeline.yml`	The nightly recipe: run everything, commit data, deploy
`dashboard/`	Static site: index.html, docs.html, assets, data/*.json
`Docs/`	This document + its HTML source

Receipts

Fact	Value
Fact rows	919 and growing daily (10 coins × up to 90 days)
Dimensions	dim_coins 50 rows · dim_date grows itself (90 → 92 during build)
Tests	15, all passing, every night
Schedule	01:13 UTC = 06:43 IST daily + manual button
Infrastructure cost	$0 — Snowflake trial, free Actions minutes, free Netlify, free API
Build time	3 days (of a 16-day plan)

Reproduce it

```
git clone https://github.com/vinn27/crypto-market-elt
# free Snowflake trial (email only) → copy .env.example to .env and fill it
python pipelines/setup_snowflake.py          # database, schemas, warehouse governance
python pipelines/extract_load.py             # fetch + land raw
cd dbt && dbt build                          # transform + 15 tests
python pipelines/export_dashboard_data.py    # dashboard JSON
```

Study companion for github.com/vinn27/crypto-market-elt · written 12 Oct 2026 · dashboard: crypto-market-elt.netlify.app · docs: crypto-market-elt.netlify.app/docs.html